

As of **17 August 2026**, the latest stable releases are **Spring Framework 7.0.8** and **Spring Boot 4.1.0**. Spring Boot 4.1.0 was released on June 10, 2026, while Spring Framework 7.0.8 is the current stable Framework release. ([Home](#))

The important point is that Spring annotations are **not normally executed in a single simple linear order**. A real Spring Boot application has a lifecycle: **startup → configuration → bean creation → dependency injection → initialization → request processing → transaction/business processing → shutdown → testing/production concerns**.

Below is the practical **project-start-to-project-end sequence** you can use when learning or designing a Spring Boot application.

1. Overall Spring Boot Annotation Sequence

A typical enterprise REST application can be understood in this order:

Sequence	Project phase	Main annotations	Main purpose
1	Project bootstrap	@SpringBootApplication	Starts Spring Boot
2	Package scanning	@ComponentScan	Finds Spring components
3	Configuration	@Configuration	Defines configuration
4	Bean creation	@Bean	Creates explicit beans
5	Component registration	@Component, @Service, @Repository, @Controller, @RestController	Registers application classes as beans
6	Dependency injection	@Autowired, @Qualifier, @Primary	Connects dependencies
7	Configuration injection	@Value, @ConfigurationProperties	Loads application configuration
8	Bean lifecycle	@PostConstruct, @PreDestroy	Initialization/destruction
9	Web endpoint mapping	@RequestMapping, @GetMapping, @PostMapping, etc.	Maps HTTP requests
10	Request data binding	@PathVariable, @RequestParam, @RequestBody,	Reads request data

		@RequestHeader	
11	Validation	@Valid, @Validated	Validates input
12	Exception handling	@ExceptionHandler, @ControllerAdvice, @RestControllerAdvice	Handles errors
13	Business logic	@Service	Executes business operations
14	Persistence	@Repository and persistence annotations	Database interaction
15	Transactions	@Transactional	Transaction boundary
16	Cross-cutting logic	@Aspect, @Around, @Before, @After	AOP
17	Security	@PreAuthorize, @Secured, etc.	Authorization
18	Events	@EventListener, @TransactionalEventListener	Event processing
19	Scheduling	@Scheduled	Background jobs
20	Async execution	@Async	Asynchronous execution
21	Testing	@SpringBootTest, @WebMvcTest, etc.	Testing
22	Shutdown	@PreDestroy	Cleanup

The exact runtime order varies depending on the application and configuration.

2. Phase 1 — Project Entry Point

@SpringBootApplication

This is normally the **first annotation you encounter** in a Spring Boot project.

```
@SpringBootApplication
public class EmployeeApplication {
```

```

    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class, args);
    }
}

```

Conceptually:

```

@SpringBootApplication
|
+--- @SpringBootConfiguration
|
+--- @EnableAutoConfiguration
|
+--- @ComponentScan

```

So one annotation activates three major mechanisms.

Annotation	Role
@SpringBootConfiguration	Identifies the main Boot configuration
@EnableAutoConfiguration	Lets Boot configure infrastructure automatically
@ComponentScan	Searches for application components

Important

Put the main class at an appropriate root package:

```

com.example.employee
|
├── EmployeeApplication.java
|
├── controller
├── service
├── repository
├── entity
└── config

```

This allows component scanning to discover the application components.

3. Phase 2 — Component Scanning

@ComponentScan

@Configuration

```
@ComponentScan("com.example.employee")
public class AppConfig {
}
```

Spring searches the specified packages for classes such as:

```
@Component
@Service
@Repository
@Controller
@RestController
```

Example:

```
@Service
public class EmployeeService {
}
```

Spring discovers this class and registers it as a bean.

Flow

```
@ComponentScan
↓
Find classes
↓
@Component
@Service
@Repository
@Controller
@RestController
↓
Register beans
```

4. Phase 3 — Configuration

@Configuration

Used to define Spring configuration classes.

```
@Configuration
public class AppConfig {

}
```

Example:

```
@Configuration
```

```
public class DatabaseConfig {  
  
    @Bean  
    public EmployeeService employeeService() {  
        return new EmployeeService();  
    }  
}
```

Think of:

```
@Configuration  
↓  
Configuration class  
↓  
@Bean methods  
↓  
Spring beans
```

5. Phase 4 — Explicit Bean Creation

@Bean

```
@Configuration  
public class AppConfig {  
  
    @Bean  
    public EmployeeService employeeService() {  
        return new EmployeeService();  
    }  
}
```

Spring executes the configuration method and registers the returned object in the application context.

Conceptually:

```
@Bean  
↓  
Object created  
↓  
Spring IoC Container  
↓  
Bean registered
```

When should you use @Bean?

Use it particularly when:

- you need to configure a third-party class;
- you need custom construction;
- you need special bean configuration.

Example:

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper();
}
```

6. Phase 5 — Component Registration

There are several stereotype annotations.

Annotation	Typical layer	Meaning
@Component	Generic	General Spring-managed component
@Service	Business layer	Business/service component
@Repository	Data layer	Repository/data-access component
@Controller	MVC web layer	MVC controller
@RestController	REST API	REST controller

@Component

```
@Component
public class EmailUtility {

    public void sendEmail() {
        System.out.println("Email sent");
    }
}
```

@Service

```
@Service
public class EmployeeService {

    public Employee findEmployee(Long id) {
```

```
        // business logic
        return new Employee();
    }
}
```

@Repository

```
@Repository
public class EmployeeRepository {

    public Employee findById(Long id) {
        // database operation
        return new Employee();
    }
}
```

@RestController

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

}
```

7. Phase 6 — Dependency Injection

After Spring discovers beans, it needs to connect them.

The major annotations are:

```
@Autowired
@Qualifier
@Primary
@Inject
@Resource
```

Spring's current documentation supports these injection mechanisms, including @Autowired, @Qualifier, @Value, @Resource, @Inject, and @Named. ([Home](#))

Constructor Injection — Recommended

```
@Service
public class EmployeeService {
```

```
private final EmployeeRepository repository;

public EmployeeService(EmployeeRepository repository) {
    this.repository = repository;
}
}
```

With a single constructor, Spring can use it without explicitly writing `@Autowired`.

You can also write:

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    @Autowired
    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }
}
```

8. Multiple Beans — `@Qualifier`

Suppose:

```
@Component
public class EmailNotification implements Notification {
}
```

and:

```
@Component
public class SmsNotification implements Notification {
}
```

Now:

```
@Service
public class NotificationService {

    private final Notification notification;

    public NotificationService(
        @Qualifier("emailNotification")
        Notification notification) {

        this.notification = notification;
    }
}
```

```
}  
}
```

@Qualifier tells Spring **which bean should be injected**.

9. Multiple Beans — @Primary

```
@Component  
@Primary  
public class EmailNotification implements Notification {  
}
```

Now if multiple implementations exist:

```
public NotificationService(Notification notification) {  
    this.notification = notification;  
}
```

Spring prefers the @Primary bean.

10. Phase 7 — Configuration Properties

Spring Boot applications commonly read:

```
application.properties  
application.yml
```

For example:

```
employee.company-name=ABC Technologies  
employee.max-employees=500
```

You can bind these using @ConfigurationProperties.

```
@ConfigurationProperties(prefix = "employee")  
public class EmployeeProperties {  
  
    private String companyName;  
    private int maxEmployees;  
  
    // getters and setters  
}
```

This is preferable to scattering many individual @Value expressions throughout a large application.

11. @Value

For individual values:

```
@Value("${employee.company-name}")
private String companyName;
```

Example:

```
@Service
public class EmployeeService {

    @Value("${employee.company-name}")
    private String companyName;

}
```

Conceptually:

```
application.properties
↓
Spring Environment
↓
@Value
↓
Java field
```

12. Phase 8 — Bean Initialization

Once dependencies are injected, initialization callbacks can execute.

@PostConstruct

```
@Service
public class EmployeeService {

    @PostConstruct
    public void initialize() {
        System.out.println("EmployeeService initialized");
    }

}
```

Typical conceptual lifecycle:

```
Bean instantiated
↓
```

Dependencies injected



@PostConstruct



Bean ready

13. Bean Destruction

@PreDestroy

@Service

public class EmployeeService {

 @PreDestroy

 public void cleanup() {

 System.out.println("Cleaning resources");

 }

}

Conceptually:

Application shutdown



@PreDestroy



Bean destroyed

Spring's testing documentation also recognizes @PostConstruct and @PreDestroy as lifecycle annotations for application components. ([Home](#))

14. Phase 9 — REST Controller

Now the application is ready to receive HTTP requests.

@RestController

@RequestMapping("/employees")

public class EmployeeController {

}

@RequestMapping

Defines a base or general request mapping.

@RequestMapping("/employees")

15. HTTP Method Mapping

Modern Spring applications commonly use:

Annotation	HTTP method
@GetMapping	GET
@PostMapping	POST
@PutMapping	PUT
@PatchMapping	PATCH
@DeleteMapping	DELETE

Example:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return service.findEmployee(id);
    }

    @PostMapping
    public Employee createEmployee(
        @RequestBody Employee employee) {
        return service.save(employee);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(
        @PathVariable Long id,
        @RequestBody Employee employee) {
        return service.update(id, employee);
    }

    @DeleteMapping("/{id}")
    public void deleteEmployee(@PathVariable Long id) {
        service.delete(id);
    }
}
```

16. Phase 10 — Request Data Extraction

Spring provides several annotations for obtaining HTTP request information.

Annotation	Gets
@PathVariable	URL path value
@RequestParam	Query parameter
@RequestBody	Request body
@RequestHeader	HTTP header
@CookieValue	Cookie
@RequestAttribute	Request attribute

Example URL:

GET /employees/100?department=IT

Code:

```
@GetMapping("/{id}")
public Employee getEmployee(
    @PathVariable Long id,
    @RequestParam String department) {

    return service.find(id, department);
}
```

17. Phase 11 — Validation

For REST applications, request validation is normally performed before business processing.

```
@PostMapping
public Employee create(
    @Valid @RequestBody EmployeeRequest request) {

    return service.create(request);
}
```

DTO:

```
public class EmployeeRequest {

    @NotBlank
    private String name;
```

```
@Email
private String email;

@Min(18)
private int age;
}
```

Common validation annotations include:

```
@NotNull
@NotBlank
@NotEmpty
@Size
@Min
@Max
@Positive
@Negative
@email
@Pattern
```

@Valid triggers validation of the request object.

18. @Validated

@Validated is useful especially for validation groups and method-level validation.

```
@Service
@Validated
public class EmployeeService {

}
```

For example:

```
public void update(
    @Min(1) Long employeeId) {

}
```

19. Phase 12 — Controller → Service

The request should normally flow:

```
Client
↓
```

Controller
↓
Service
↓
Repository
↓
Database

Example:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return service.findEmployee(id);
    }
}
```

20. Phase 13 — Service Layer

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public Employee findEmployee(Long id) {
        return repository.findById(id);
    }
}
```

@Service does not itself execute business logic automatically.

It tells Spring:

"Manage this class as a Spring bean."

21. Phase 14 — Repository/Data Layer

For Spring Data JPA applications, you may have:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

}
```

Spring Data creates the repository implementation.

You may also use:

`@Repository`

for custom repository implementations.

22. Phase 15 — Entity Mapping

For JPA entities, annotations generally belong to the persistence model rather than the Spring Framework itself.

Example:

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}
```

Common JPA annotations:

Annotation	Purpose
<code>@Entity</code>	Persistent entity
<code>@Table</code>	Database table
<code>@Id</code>	Primary key

@GeneratedValue	ID generation
@Column	Column mapping
@OneToOne	One-to-one relationship
@OneToMany	One-to-many relationship
@ManyToOne	Many-to-one relationship
@ManyToMany	Many-to-many relationship

23. Phase 16 — Transaction Boundary

@Transactional

This is extremely important in enterprise applications.

```
@Service
public class EmployeeService {

    @Transactional
    public void transferEmployee(
        Long employeeId,
        Long departmentId) {

        // multiple database operations
    }
}
```

Conceptually:

```
@Transactional
↓
Transaction begins
↓
Operation 1
↓
Operation 2
↓
Operation 3
↓
Success → COMMIT
Failure → ROLLBACK
```

It is generally better to place transaction boundaries around **service-layer business operations** rather than randomly annotating every repository method.

24. Phase 17 — Exception Handling

Instead of putting error handling into every controller:

```
try {  
    ...  
}  
catch (...) {  
    ...  
}
```

you can use:

@ExceptionHandler

```
@RestController  
@RequestMapping("/employees")  
public class EmployeeController {  
  
    @ExceptionHandler(EmployeeNotFoundException.class)  
    public String handleEmployeeNotFound(  
        EmployeeNotFoundException ex) {  
  
        return ex.getMessage();  
    }  
}
```

25. Global Exception Handling

@RestControllerAdvice

For an entire application:

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(EmployeeNotFoundException.class)  
    public ResponseEntity<String> handle(  
        EmployeeNotFoundException ex) {  
  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(ex.getMessage());  
    }  
}
```

```
}  
}
```

Conceptually:

```
Controller  
↓  
Exception  
↓  
@RestControllerAdvice  
↓  
@ExceptionHandler  
↓  
HTTP error response
```

26. Phase 18 — AOP

Spring AOP handles cross-cutting concerns.

Common examples:

Logging
Security
Transactions
Auditing
Performance measurement

Important annotations include:

@Aspect
@Before
@After
@AfterReturning
@AfterThrowing
@Around
@Pointcut

Example:

```
@Aspect  
@Component  
public class LoggingAspect {  
  
    @Before("execution(* com.example.service.*(..))")  
    public void logBefore() {  
        System.out.println("Service method called");  
    }  
}
```

@Around

```
@Around("execution(* com.example.service.*(..))")
public Object logExecution(ProceedingJoinPoint pjp)
    throws Throwable {

    long start = System.currentTimeMillis();

    Object result = pjp.proceed();

    long end = System.currentTimeMillis();

    System.out.println(
        "Execution time = " + (end - start));

    return result;
}
```

27. Phase 19 — Security

Spring Security introduces another layer around application execution.

Typical modern authorization annotations include:

```
@PreAuthorize
@PostAuthorize
@Secured
@RolesAllowed
```

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteEmployee(Long id) {
    repository.deleteById(id);
}
```

Conceptually:

```
HTTP request
↓
Security filter chain
↓
Authentication
↓
Authorization
↓
Controller
↓
```

28. Phase 20 — Events

Spring applications can communicate through application events.

Publisher

```
applicationEventPublisher.publishEvent(  
    new EmployeeCreatedEvent(employee));
```

Listener

```
@Component  
public class EmployeeEventListener {  
  
    @EventListener  
    public void handle(EmployeeCreatedEvent event) {  
        System.out.println("Employee created");  
    }  
}
```

For transaction-sensitive events:

```
@TransactionalEventListener
```

This can be useful when an event should be processed in relation to a transaction lifecycle.

29. Phase 21 — Asynchronous Processing

@Async

```
@Service  
public class EmailService {  
  
    @Async  
    public void sendEmail() {  
        // long-running operation  
    }  
}
```

Caller:

```
emailService.sendEmail();
```

The method can execute asynchronously when async processing is enabled/configured.

Conceptually:

Request thread
↓
@Async method
↓
Executor
↓
Worker thread

30. Phase 22 — Scheduled Jobs

@Scheduled

```
@Component
public class ReportScheduler {

    @Scheduled(cron = "0 0 9 * * *")
    public void generateReport() {
        System.out.println("Generating report");
    }
}
```

Another example:

```
@Scheduled(fixedRate = 60000)
public void process() {
    System.out.println("Runs periodically");
}
```

Conceptually:

Application running
↓
Scheduler
↓
@Scheduled method
↓
Background execution

31. Phase 23 — Testing

Spring provides a large set of testing annotations. The current Framework 7 documentation includes annotations such as @ContextConfiguration, @ActiveProfiles, @DynamicPropertySource, @MockitoBean, @MockitoSpyBean, @DirtiesContext, @Commit, @Rollback, and @Sql. ([Home](#))

@SpringBootTest

For a full Spring Boot application context:

```
@SpringBootTest
class EmployeeApplicationTest {

    @Test
    void contextLoads() {
    }
}
```

32. Web Layer Testing

@WebMvcTest

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {

}
```

Useful when testing the MVC/controller layer without loading the entire application.

33. Repository Testing

For JPA repository tests:

```
@DataJpaTest
class EmployeeRepositoryTest {

}
```

This focuses on persistence-related components.

34. Phase 24 — Profiles

Different environments often require different configuration.

```
@Profile("dev")
@Component
public class DevelopmentService {

}
```

Example:

application.properties

application-dev.properties
application-test.properties
application-prod.properties

Then:

spring.profiles.active=dev

Conceptually:

Environment
↓
Active Profile
↓
@Profile
↓
Appropriate beans

35. Phase 25 — Conditional Configuration

Spring Boot uses conditional configuration extensively.

Common annotations include:

@ConditionalOnClass
@ConditionalOnMissingBean
@ConditionalOnBean
@ConditionalOnProperty
@ConditionalOnMissingClass
@ConditionalOnResource

Example:

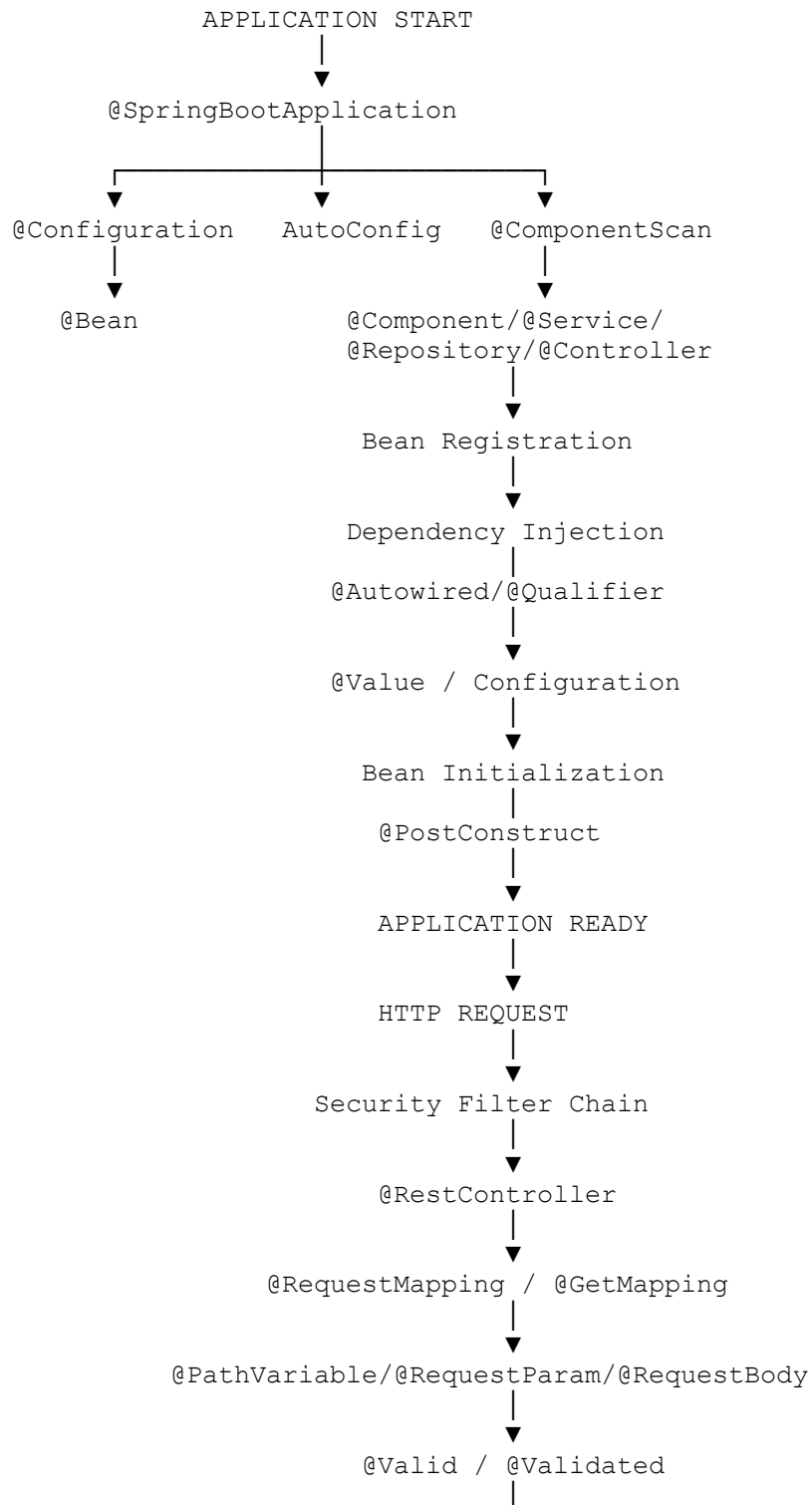
```
@Configuration
@ConditionalOnProperty(
    name = "feature.email.enabled",
    havingValue = "true"
)
public class EmailConfiguration {

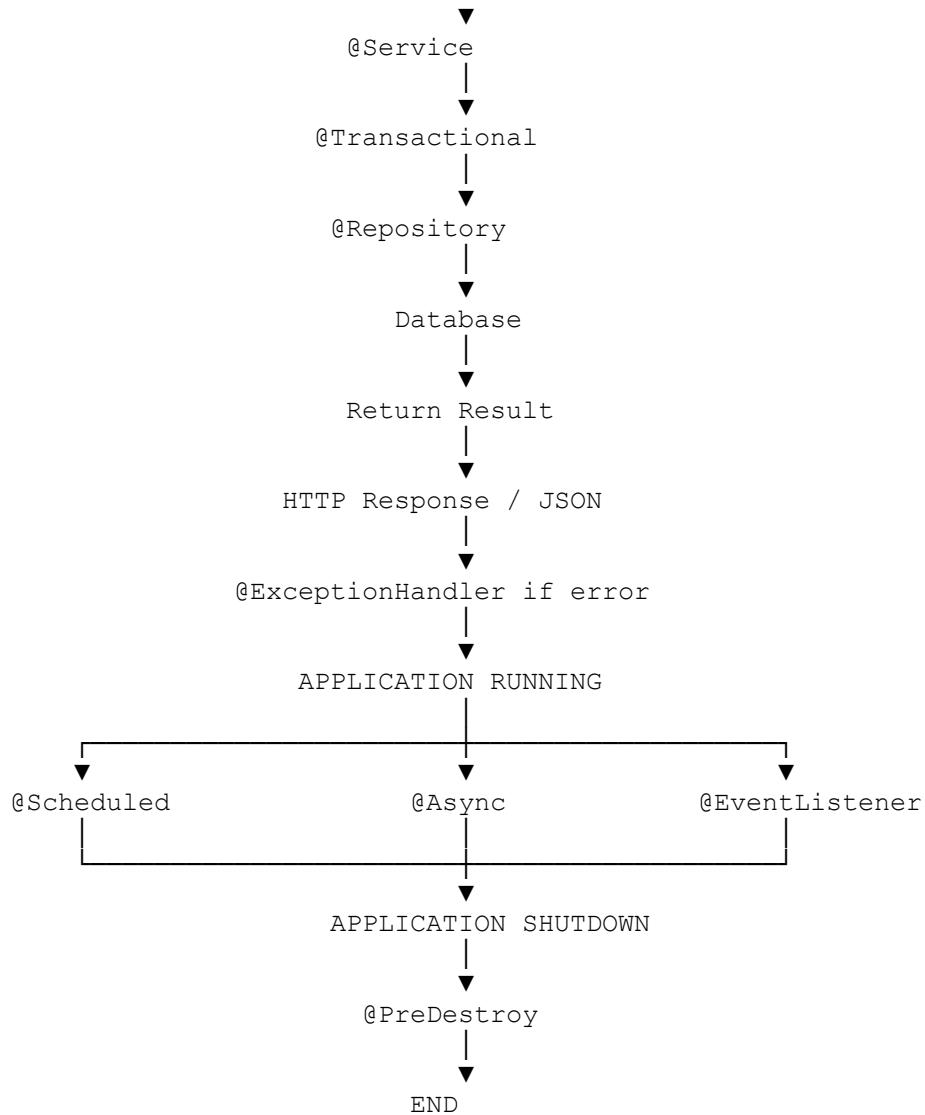
}
```

This is one of the mechanisms behind Spring Boot's auto-configuration model.

36. Complete End-to-End Runtime Picture

A useful mental model is:





37. Complete Annotation Sequence Table

#	Phase	Annotation	Example	Main responsibility
1	Startup	@SpringBootApplication	@SpringBootApplication	Bootstraps application
2	Configuration	@SpringBootConfiguration	Usually indirectly used	Boot configuration
3	Auto configuration	@EnableAutoConfiguration	Usually inside Boot annotation	Automatic infrastructure

4	Scanning	@ComponentScan	@ComponentScan("com.app")	Finds components
5	Configuration	@Configuration	@Configuration	Configuration class
6	Bean creation	@Bean	@Bean DataSource ds()	Creates bean
7	Generic component	@Component	@Component	Registers bean
8	Business layer	@Service	@Service	Business component
9	Data layer	@Repository	@Repository	Data-access component
10	MVC	@Controller	@Controller	MVC controller
11	REST	@RestController	@RestController	REST controller
12	DI	@Autowired	Constructor/field/parameter	Dependency injection
13	DI selection	@Qualifier	@Qualifier("email")	Selects bean
14	DI priority	@Primary	@Primary	Preferred bean
15	Configuration	@Value	@Value("\${app.name}")	Injects property
16	Configuration	@ConfigurationProperties	@ConfigurationProperties	Property binding
17	Lifecycle	@PostConstruct	init()	Initialization
18	Lifecycle	@PreDestroy	cleanup()	Shutdown
19	Mapping	@RequestMapping	/employees	Request mapping
20	GET	@GetMapping	/employees/{id}	GET endpoint
21	POST	@PostMapping	/employees	POST endpoint
22	PUT	@PutMapping	/employees/{id}	PUT endpoint
23	PATCH	@PatchMapping	/employees/{id}	PATCH endpoint

24	DELETE	@DeleteMapping	/employees/{id}	DELETE endpoint
25	Path	@PathVariable	@PathVariable Long id	URL variable
26	Query	@RequestParam	@RequestParam String name	Query parameter
27	Body	@RequestBody	@RequestBody Employee e	JSON/body binding
28	Header	@RequestHeader	@RequestHeader String token	Header
29	Cookie	@CookieValue	@CookieValue String id	Cookie
30	Validation	@Valid	@Valid @RequestBody	Object validation
31	Validation	@Validated	@Validated	Validation groups/method validation
32	Exception	@ExceptionHandler	@ExceptionHandler(...)	Handles exception
33	Global exception	@ControllerAdvice	@ControllerAdvice	Global MVC error handling
34	Global REST exception	@RestControllerAdvice	@RestControllerAdvice	Global REST error handling
35	Transaction	@Transactional	@Transactional	Transaction boundary
36	AOP	@Aspect	@Aspect	Aspect definition
37	AOP	@Pointcut	@Pointcut(...)	Defines join points
38	AOP	@Before	@Before(...)	Before method
39	AOP	@After	@After(...)	After method
40	AOP	@AfterReturning	@AfterReturning(...)	Successful return

41	AOP	@AfterThrowing	@AfterThrowing(...)	Exception
42	AOP	@Around	@Around(...)	Wraps method
43	Security	@PreAuthorize	@PreAuthorize(...)	Authorization
44	Security	@PostAuthorize	@PostAuthorize(...)	Post-invocation authorization
45	Events	@EventListener	@EventListener	Event listener
46	Events	@TransactionalEventListener	@TransactionalEventListener	Transaction-aware event
47	Async	@Async	@Async	Async execution
48	Scheduling	@Scheduled	@Scheduled(...)	Scheduled task
49	Profiles	@Profile	@Profile("prod")	Environment-specific bean
50	Conditions	@ConditionalOnProperty	@ConditionalOnProperty(...)	Conditional configuration
51	Testing	@SpringBootTest	@SpringBootTest	Full integration context
52	MVC testing	@WebMvcTest	@WebMvcTest	Web-layer test
53	JPA testing	@DataJpaTest	@DataJpaTest	Repository/JPA test
54	Shutdown	@PreDestroy	cleanup()	Final cleanup

38. The Most Important Concept: Annotation Categories

Do **not** try to memorize Spring annotations as one giant list.

A much better approach is to group them:

Category	Learn these first
Application startup	@SpringBootApplication
Configuration	@Configuration, @Bean, @ConfigurationProperties

Component scanning	@Component, @Service, @Repository, @Controller, @RestController
Dependency injection	@Autowired, @Qualifier, @Primary
Lifecycle	@PostConstruct, @PreDestroy
REST	@RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
Request binding	@PathVariable, @RequestParam, @RequestBody, @RequestHeader
Validation	@Valid, @Validated
Exception handling	@ExceptionHandler, @ControllerAdvice, @RestControllerAdvice
Database	@Repository, JPA annotations
Transaction	@Transactional
AOP	@Aspect, @Around, @Before, etc.
Security	@PreAuthorize, @PostAuthorize
Events	@EventListener, @TransactionalEventListener
Async	@Async
Scheduling	@Scheduled
Environment	@Profile, conditional annotations
Testing	@SpringBootTest, @WebMvcTest, @DataJpaTest

39. A Complete Small Project

Here is how these concepts fit together.

Main class

```
@SpringBootApplication
public class EmployeeApplication {
```

```
public static void main(String[] args) {  
    SpringApplication.run(EmployeeApplication.class, args);  
}  
}
```

Entity

```
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue  
    private Long id;  
  
    private String name;  
  
    private String email;  
}
```

Repository

```
@Repository  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Long> {  
}
```

Service

```
@Service  
public class EmployeeService {  
  
    private final EmployeeRepository repository;  
  
    public EmployeeService(EmployeeRepository repository) {  
        this.repository = repository;  
    }  
  
    @Transactional  
    public Employee save(Employee employee) {  
        return repository.save(employee);  
    }  
  
    public Employee find(Long id) {  
        return repository.findById(id)  
            .orElseThrow(  
                () -> new EmployeeNotFoundException(id)  
            );  
    }  
}
```

```
@PostConstruct
public void initialize() {
    System.out.println("EmployeeService initialized");
}
}
```

Controller

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Employee get(
        @PathVariable Long id) {

        return service.find(id);
    }

    @PostMapping
    public Employee create(
        @Valid @RequestBody Employee employee) {

        return service.save(employee);
    }
}
```

Global exception handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<String> handle(
        EmployeeNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

40. What Actually Happens When a Request Arrives?

Suppose the client sends:

POST /employees
Content-Type: application/json

with:

```
{  
  "name": "Anurag",  
  "email": "anurag@example.com"  
}
```

The conceptual sequence is:

```
1. Application already started  
   ↓  
2. Spring Security filters  
   ↓  
3. DispatcherServlet  
   ↓  
4. @RestController  
   ↓  
5. @PostMapping  
   ↓  
6. @RequestBody  
   ↓  
7. JSON → Java object  
   ↓  
8. @Valid  
   ↓  
9. Controller method  
   ↓  
10. @Service  
   ↓  
11. @Transactional  
   ↓  
12. @Repository  
   ↓  
13. JPA/Hibernate  
   ↓  
14. Database  
   ↓  
15. Transaction COMMIT  
   ↓  
16. Java object returned  
   ↓  
17. JSON serialization  
   ↓  
18. HTTP response
```

If an exception occurs:

```

Exception
↓
@RestControllerAdvice
↓
@ExceptionHandler
↓
Error response

```

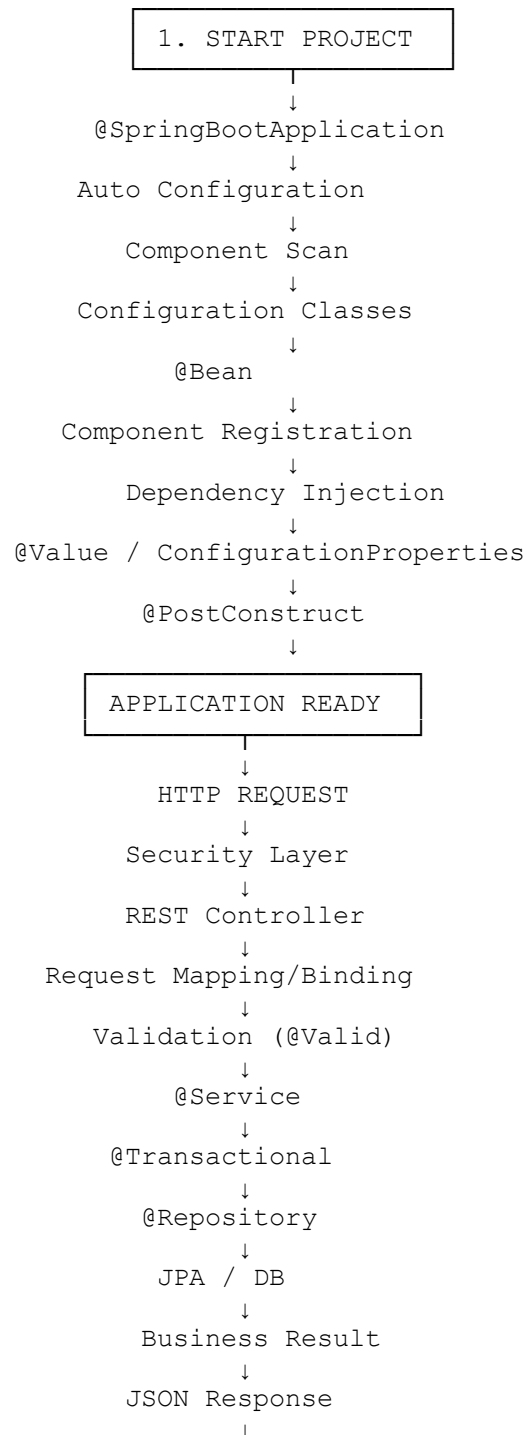
41. Startup vs Request-Time vs Shutdown

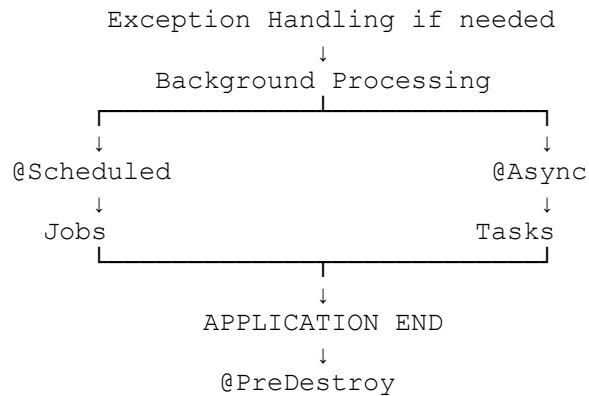
One of the most important distinctions is that **not all annotations participate at the same time**.

Time	Important annotations
Application startup	@SpringBootApplication, @Configuration, @ComponentScan, @Bean, @Component, @Service, @Repository, @Controller
Bean creation	@Autowired, @Qualifier, @Primary, @Value, @ConfigurationProperties
Bean initialization	@PostConstruct
HTTP request	@RestController, @RequestMapping, @GetMapping, @PostMapping, @PathVariable, @RequestBody, @RequestParam
Validation	@Valid, @Validated
Security	@PreAuthorize, etc.
Business processing	@Service
Transaction processing	@Transactional
Database	@Repository, JPA annotations
AOP	@Aspect, @Around, @Before, etc.
Exception	@ExceptionHandler, @RestControllerAdvice
Background	@Scheduled, @Async
Events	@EventListener, @TransactionalEventListener
Shutdown	@PreDestroy
Testing	@SpringBootTest, @WebMvcTest, @DataJpaTest, etc.

42. Final Mental Model

For an enterprise Spring Boot application, remember this sequence:





Version note

Spring Framework 7 is the foundation generation for Spring Boot 4 and focuses on the modern Java/Jakarta ecosystem. Spring Framework 7 officially targets **Java 25 as the latest LTS while retaining Java 17 as the baseline**, and uses Jakarta EE 11 APIs. ([Home](#)) Spring Boot 4.1.0 is currently the latest stable Boot release, while 4.2.0 is currently a snapshot. ([Home](#))

For learning **Spring Boot 4.1 + Spring Framework 7**, therefore, the most useful progression is:

@SpringBootApplication → configuration/beans → component scanning → DI → lifecycle → REST → validation → service → transactions → repository/database → exception handling → AOP/security → events/async/scheduling → testing → shutdown.

[Spring Framework 7.0 Reference Documentation](#)

[Spring Boot 4.1 Reference Documentation](#)